

Huffman

December 13, 2025

```
[16]: def huffman_encode_activations(input_file):  
    """  
    Read activation values from file, skipping first 2 comment lines  
    and processing 64-bit lines into 8-bit values  
    """  
    # Read the activation data file, skipping first 2 comment lines  
    with open(input_file, 'r') as f:  
        lines = f.readlines()  
  
    # Skip first 2 lines (comments) and process the rest  
    data_lines = lines[2:]  
  
    # Process each line as 64-bit data containing 8 bytes  
    line_data = []  
    for line in data_lines:  
        line = line.strip() # Remove newline characters  
        if len(line) == 64: # Ensure it's a valid 64-bit line  
            # Split the 64-bit line into 8 bytes (8 bits each)  
            bytes_in_line = []  
            for i in range(8):  
                byte_str = line[i*8:(i+1)*8]  
                byte_val = int(byte_str, 2) # Convert binary string to integer  
                bytes_in_line.append(byte_val)  
            line_data.append(bytes_in_line)  
  
    print(f"Total lines processed: {len(line_data)}")  
    return line_data
```

```
[17]: def huffman_encode_per_line(line_data):  
    """  
    Huffman encoding for activation values:  
    - 0 values: store as '0' (1 bit)  
    - non-0 values: store as '0' + 8-bit binary representation (9 bits)  
    """  
    encoded_lines = []  
    for line_bytes in line_data:  
        encoded_bits = ""  
        #print line_bytes
```

```

    print(f"Encoding line: {line_bytes}")

    for byte_val in line_bytes:
        if byte_val == 0:
            # 0 encodes to 1 bit: '0'
            encoded_bits += "0"
        else:
            # Other values encode to 9 bits: '1' + 8 bits of original data
            encoded_bits += "1" + format(byte_val, '08b')
    encoded_lines.append(encoded_bits)
    return encoded_lines

```

```

[19]: # Run encoding
line_data = huffman_encode_activations('activation4b4b.txt')
encoded_lines = huffman_encode_per_line(line_data)

# Calculate compression ratio
total_bytes = sum(len(line) for line in line_data)
original_size_bits = total_bytes * 8 # Original size in bits
compressed_size_bits = sum(len(line) for line in encoded_lines) # Compressed
    ↪ size in bits
compression_ratio = original_size_bits / compressed_size_bits if
    ↪ compressed_size_bits > 0 else 0

print(f"Original size: {original_size_bits} bits")
print(f"Compressed size: {compressed_size_bits} bits")
print(f"Compression ratio: {compression_ratio:.2f}")

# Save result to output file, maintaining line structure
with open('activation4b4b_huffman.txt', 'w') as f:
    # Write the same header comments
    # f.write("#time0row7[msb-lsb],time0row6[msb-lst],....
    ↪ ,time0row0[msb-lst]#\n")
    # f.write("#time1row7[msb-lsb],time1row6[msb-lst],....
    ↪ ,time1row0[msb-lst]#\n")

    # Write each encoded line
    for encoded_line in encoded_lines:
        f.write(encoded_line + '\n')

print("Encoded data saved to 'activation4b4b_huffman.txt' with preserved line
    ↪ structure")

```

```

Total lines processed: 36
Encoding line: [0, 0, 0, 0, 0, 0, 0, 0, 0]
Encoding line: [0, 0, 0, 0, 0, 0, 0, 0, 0]
Encoding line: [0, 0, 0, 0, 0, 0, 0, 0, 0]

```

```

Encoding line: [0, 0, 0, 0, 0, 0, 0, 0]
Encoding line: [0, 0, 0, 0, 0, 0, 0, 0]
Encoding line: [0, 0, 0, 0, 0, 0, 0, 0]
Encoding line: [0, 0, 0, 0, 0, 0, 0, 0]
Encoding line: [0, 0, 0, 5, 0, 0, 10, 3]
Encoding line: [0, 0, 0, 7, 0, 0, 14, 0]
Encoding line: [0, 0, 2, 2, 0, 0, 10, 0]
Encoding line: [0, 0, 3, 0, 3, 0, 7, 0]
Encoding line: [0, 0, 0, 0, 0, 0, 0, 0]
Encoding line: [0, 0, 0, 0, 0, 0, 0, 0]
Encoding line: [0, 0, 1, 8, 0, 0, 13, 0]
Encoding line: [0, 0, 0, 7, 0, 0, 15, 0]
Encoding line: [0, 0, 0, 0, 0, 0, 6, 0]
Encoding line: [0, 0, 0, 0, 0, 0, 7, 0]
Encoding line: [0, 0, 0, 0, 0, 0, 0, 0]
Encoding line: [0, 0, 0, 0, 0, 0, 0, 0]
Encoding line: [0, 0, 1, 8, 0, 0, 6, 0]
Encoding line: [0, 0, 0, 10, 0, 0, 0, 0]
Encoding line: [0, 0, 0, 4, 0, 0, 0, 0]
Encoding line: [0, 0, 0, 2, 0, 0, 0, 0]
Encoding line: [0, 0, 0, 0, 0, 0, 0, 0]
Encoding line: [0, 0, 0, 0, 0, 0, 0, 0]
Encoding line: [0, 0, 1, 4, 0, 0, 2, 6]
Encoding line: [0, 0, 0, 5, 0, 0, 0, 8]
Encoding line: [0, 0, 0, 4, 0, 0, 0, 8]
Encoding line: [0, 0, 0, 4, 0, 0, 0, 5]
Encoding line: [0, 0, 0, 0, 0, 0, 0, 0]
Encoding line: [0, 0, 0, 0, 0, 0, 0, 0]
Encoding line: [0, 0, 0, 0, 0, 0, 0, 0]
Encoding line: [0, 0, 0, 0, 0, 0, 0, 0]
Encoding line: [0, 0, 0, 0, 0, 0, 0, 0]
Encoding line: [0, 0, 0, 0, 0, 0, 0, 0]
Original size: 2304 bits
Compressed size: 560 bits
Compression ratio: 4.11
Encoded data saved to 'activation4b4b_huffman.txt' with preserved line structure

```